

CHAPTER 4 SYSTEM DESIGN

Introduction

Before looking at any single screen or service, it helps to see the whole system at once. RakshaSafe is not one big program but a set of focused modules that each own a small, well-defined job: signing users in, recording emergencies, attaching real locations, keeping emergency contacts, organising facilities and rescue teams, sending notifications, scoring risk and letting administrators supervise everything. This chapter walks through that design the way the software is actually built, so every section below traces back to real pages, real API routes and real database collections.

The table below lists every module of the system with what it covers and where it lives in the implementation. It is the map for the rest of the chapter: Section 4.1 explains how the modules work together, Section 4.2 describes each interface in detail, and Section 4.3 covers deployment and security.

Module	What it covers	Main screens / API
Authentication	Registration, login, logout and JWT sessions for all three roles	Login, Register; /api/auth/*
User Management	Profiles, language preference and account state	Profile; /api/auth/me, /api/auth/profile
SOS / Incident Management	Emergency reporting, status workflow, history and ownership	SOS, Incident Detail, Dashboard; /api/incidents/*
Location	Coordinate capture, stored points and readable addresses	SOS location step, Location cards; /api/geocode/*
Emergency Contacts	Per-user contacts with call links and alert channels	Emergency Contacts, Admin Emergency Contacts; /api/emergency-contacts/*, /api/admin/emergency-contacts
Emergency Resources	Searchable directory and nearby lookup of help	Resources, Nearby sections; /api/facilities, /api/nearby-resources, /api/nearby
Facilities	Hospitals, shelters, police and fire stations, relief centres	Resources, Admin Facilities; /api/facilities, /api/admin/facilities
Rescue Teams	Registered teams with type, contact, members and base	Resources, Admin Teams; /api/rescue-teams, /api/admin/rescue-teams
Rescue Assignments	Incident-to-team assignment records and their lifecycle	Incident Detail, Admin incident detail; /api/admin/incidents/:id/assignments, /api/responder/assignments
Unsafe-Area Reporting	User reports with admin verification queue	Unsafe Reports, Admin Unsafe Reports; /api/unsafe-reports/*, /api/admin/unsafe-reports
Risk Zones	Administrator-maintained boundary records with risk levels	Admin masters; RiskZones collection
Risk Assessment (AI)	Deterministic raksha-risk-v1 scoring of incident locations	Risk panels; /api/incidents/:id/risk, FastAPI POST /risk/assess
Notifications	In-app and provider-channel delivery records with status	Notifications; /api/notifications, /api/responder/notifications
Weather Enrichment	Informational current weather for incident coordinates	Weather cards; /api/weather (Open-Meteo, no key)

Administration	Dashboard, users, reports, activity logs and analytics	Admin Dashboard, Users, Reports; /api/admin/*
----------------	--	---

4.1 System Architecture

The system design of RakshaSafe – Women Safety and Disaster Emergency Response System describes how the different parts of the system work together to record emergency incidents, associate them with real locations, organise emergency resources, support rescue assignments, produce assistive risk assessments, deliver notifications and provide administration. The design keeps three concerns separate: the React web application that users and administrators operate, the Node.js backend that enforces authentication, validation and record relationships, and the Python AI service that scores risk. MongoDB stores every persistent record, and the backend is the only writer of application data.

4.1.1 Overall System Working

The overall working of RakshaSafe is divided into the following stages:

1. User Authentication

a. A new user registers with name, email, phone and password. An existing user, administrator or responder signs in with the registered email or phone number together with the password. b. Passwords are never stored in plain text; they are hashed with bcrypt and verified on login. c. On successful authentication the backend issues a JSON Web Token (JWT). The frontend stores the session and redirects by role: USER accounts reach the user dashboard, ADMIN accounts reach the admin dashboard, and RESPONDER accounts reach the responder workspace.

2. SOS and Incident Creation

a. The signed-in user opens the SOS page and describes the emergency: incident type (Safety or Disaster), a category such as Women Safety, Medical Emergency, Accident, Fire, Flood, Earthquake or Other Emergency, a free-text description and a priority of LOW, MEDIUM, HIGH or CRITICAL. b. The backend validates the request, creates an Incidents record with status REPORTED and returns the stored incident. The incident immediately appears in the user dashboard and in the admin incident list.

3. Location Capture and Association

a. Where the user grants permission, the browser Geolocation API supplies latitude, longitude and accuracy. The coordinates are sent with the incident request and stored as a Locations record linked through locationId. b. A server-side reverse-geocode step resolves a readable address, city, district, state, postal code and country for display. If permission is denied or the position is unavailable, the incident is still recorded without coordinates; no false location is ever assumed or stored.

4. Emergency Resource Lookup

a. Around a real query point the system lists nearby Facilities (Hospital, Shelter, Police Station, Fire Station, Relief Centre) and active RescueTeams, nearest first with straight-line distances. b. Each card shows the values actually present in the source record: name, type, address or “Address unavailable”, and distance. A Call button using a tel: link is rendered only when a real phone number exists, and a Directions link is built only from real coordinates. Where an external places provider is configured, its live results are merged with stored records; when it is unavailable the page honestly reports that and continues with stored resources.

5. Rescue Assignment

a. An administrator assigns a registered, active rescue team to an incident, optionally with notes. b. Assignment status follows ASSIGNED → EN_ROUTE → ON_SCENE → COMPLETED, with CANCELLED where applicable. Every change is a software record in RescueAssignments; the system never claims automatic dispatch, guaranteed response or an exact arrival time.

6. AI Risk Assessment

a. For an incident that has stored coordinates, the user (or the admin view of the same incident) can request an assistive risk assessment. The backend assembles fixed factors — priority, incident type, counts of verified and unverified unsafe-area reports nearby, and other active incidents nearby — and posts them to the FastAPI AI service. b. The deterministic raksha-risk-v1 model returns a risk score from 0 to 100, a risk level of LOW, MEDIUM, HIGH or CRITICAL, the model version, the input factors and the assessment time. The backend validates this response and stores it in RiskAssessments, then serves the stored record to both user and admin views. The result is decision support only and is always shown with a disclaimer.

7. Notifications

a. Incident lifecycle events — creation, acknowledgement, assignment, status changes, resolution and cancellation — create Notification records for the incident owner and, where configured, for emergency-contact channels. b. Each notification carries its channel (Email, SMS, WhatsApp or In-App), a status of QUEUED, SENT, DELIVERED, FAILED, NOT_CONFIGURED or UNAVAILABLE, the provider response, attempt counts and timing. Delivery depends on the configured providers; a delivery failure never corrupts the incident record.

8. Administration and Reporting

a. Administrators work through the admin dashboard: users, emergency contacts of any user, incidents with reporter, location, history and assignments, facilities, rescue teams and their members, unsafe-area report verification, risk-zone records, generated reports in PDF, CSV or JSON, and the admin activity log. b. Incident status follows REPORTED → ACKNOWLEDGED → ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED, with CANCELLED where applicable, and each transition is stored as an IncidentUpdates record with an optional comment.

Figure 4.1: RakshaSafe System Architecture

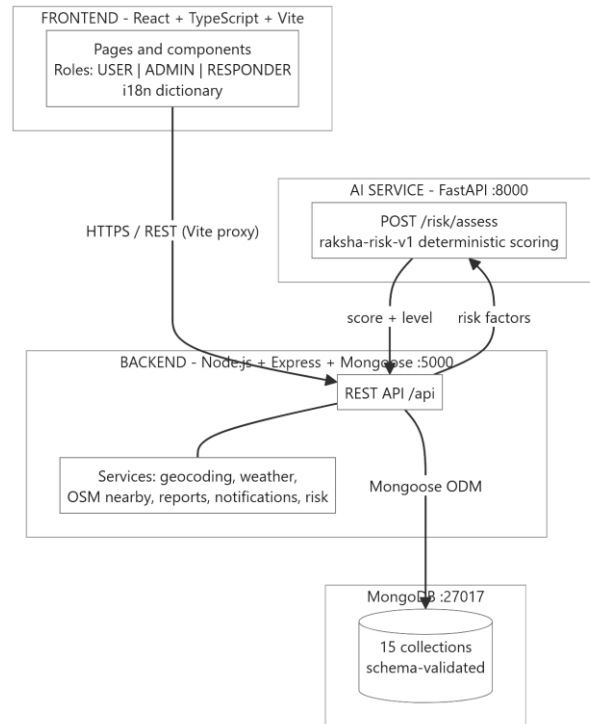


Figure 4.1: RakshaSafe System Architecture

4.1.2 Technology Used

The technologies below are the ones actually used in the implementation. Each choice matches a layer of the system, and no component in the table is aspirational: the frontend is the React application in the repository, the backend is the Express API, the database is MongoDB accessed through Mongoose, and the AI service is the FastAPI application in the ai-service folder.

Component	Technology	Purpose
Frontend	React.js, TypeScript, Vite, Tailwind CSS	User and admin web interface
Backend	Node.js, Express.js, TypeScript	REST APIs and business logic
Authentication	JWT, bcrypt	Session tokens and password security
Database	MongoDB, Mongoose ODM	Persistent records and validation
AI Service	Python, FastAPI	Deterministic risk scoring
AI Model	raksha-risk-v1 (rule-based scoring)	Risk score and level, decision support only
Location	Browser Geolocation API	Latitude, longitude and accuracy capture
Maps	OpenStreetMap, Leaflet	Incident and resource map display
Communication	Configured Email / SMS / WhatsApp providers	Notification delivery where configured
Weather	Open-Meteo (no key)	Informational weather enrichment

4.1.3 Database Design

MongoDB holds fifteen collections. Relationships are ObjectId references maintained by Mongoose schemas rather than relational foreign keys, and critical integrity is enforced through schema validation. The table below names each collection with its purpose and principal references; the detailed field lists follow Chapter 3 and the model files, so only the important references are repeated here.

Collection	Purpose	Principal references
Users	Accounts and profiles	Unique email and phone
EmergencyContacts	User emergency contacts	userId → Users
Incidents	SOS and emergency records	userId → Users; locationId → Locations (optional)
Locations	Geographic points and resolved address	Standalone; referenced by incidents, facilities, reports
Notifications	Delivery records and status	incidentId → Incidents; contactId → EmergencyContacts (optional)
RescueTeams	Registered response teams and members	members → Users; locationId → Locations (optional)
RescueAssignments	Incident-to-team assignments	incidentId → Incidents; teamId → RescueTeams; assignedBy → Users
IncidentUpdates	Status history and comments	incidentId → Incidents; updatedBy → Users
UnsafeAreaReports	User-submitted unsafe-area reports	reportedBy → Users; locationId → Locations
RiskZones	Structured risk-zone records	Standalone boundary records managed by admins
RiskAssessments	Risk scores and assessment results	locationId → Locations
Facilities	Hospitals, shelters, stations, relief centres	locationId → Locations
DisasterCategories	Maintained incident categories	Referenced by category name/code
AdminLogs	Administrative activity records	adminId → Users; targetId (various)
Reports	Generated analytics records	generatedBy → Users

The relationships that matter for the design are short: a user owns incidents, contacts and unsafe-area reports; an incident optionally points at one location and accumulates updates, assignments, notifications and — through its location — risk assessments; facilities and rescue teams each point at locations; and every administrative action appends to AdminLogs. Nothing in the design requires joins across databases, because there is only one database.

4.1.4 API Design

The backend exposes a REST API under /api with a uniform JSON envelope: success responses carry a data object, while failures carry a message and, for validation problems, a field-level issues list. Every router declares its own guard chain, so the access rule of an endpoint can be read directly from the route files: public authentication endpoints stand alone, user endpoints combine requireAuth with requireDb and owner scoping, responder endpoints add requireResponder, and the whole /api/admin tree combines requireAuth, requireAdmin and requireDb. Creation endpoints for

incidents and unsafe-area reports additionally carry per-IP rate limits against flooding. The table below lists the implemented endpoints; enrichment lookups for live maps, weather and geocoding deliberately omit requireDb so informational features stay available while MongoDB is unreachable.

Method and path	Access	Purpose
POST /api/auth/register	Public, database	Create a USER account
POST /api/auth/login	Public, database, rate-limited	Authenticate and issue a JWT
POST /api/auth/logout	Public	End the client session
GET /api/auth/me	Authenticated, database	Return the current account
PATCH /api/auth/profile	Authenticated, database	Update the own profile
POST /api/emergency-contacts	Authenticated, database, owner	Add an own emergency contact
GET /api/emergency-contacts	Authenticated, database, owner	List the own emergency contacts
PATCH /api/emergency-contacts/:id	Authenticated, database, owner	Update an own contact
DELETE /api/emergency-contacts/:id	Authenticated, database, owner	Delete an own contact
POST /api/incidents	Authenticated, database, rate-limited	Create an SOS incident
GET /api/incidents	Authenticated, database, owner	List the own incidents
GET /api/incidents/:id	Authenticated, database, owner	Incident detail with location
PATCH /api/incidents/:id	Authenticated, database, owner	Edit an own early incident
DELETE /api/incidents/:id	Authenticated, database, owner	Delete an own early incident
GET /api/incidents/:id/updates	Authenticated, database, owner	Status history of the incident
POST /api/incidents/:id/risk	Authenticated, database, owner	Request a risk assessment
GET /api/incidents/:id/risk	Authenticated, database, owner	Read stored assessments
GET /api/incidents/:id/nearby-resources	Authenticated, database, owner	Nearby help for the incident
GET /api/facilities, GET /api/facilities/:id	Authenticated, database	Operational facilities
GET /api/rescue-teams, GET /api/rescue-teams/:id	Authenticated, database	Active rescue teams
GET /api/nearby-resources	Authenticated, database	Nearby stored resources
GET /api/nearby	Authenticated, no database	Live mapped facilities, radius 1 to 5 km
GET /api/weather	Authenticated, no database	Current and today weather from Open-Meteo
GET /api/notifications	Authenticated, database, owner	Notifications of the own incidents
POST /api/unsafe-reports	Authenticated, database, rate-limited	Submit an unsafe-area report
GET /api/unsafe-reports, GET /api/unsafe-reports/:id	Authenticated, database	Read own and visible reports
PATCH /api/unsafe-reports/:id	Authenticated, database, owner	Edit an own unverified report
DELETE /api/unsafe-reports/:id	Authenticated, database, owner	Withdraw an own report
GET /api/geocode/geocode	Authenticated, no database	Forward place search within India
GET /api/geocode/reverse	Authenticated, no database	Reverse address for coordinates
GET /api/responder/assignments	Responder, database	Assignments of the caller teams
PATCH	Responder, database	Advance an assignment status

/api/responder/assignments/:id		
GET /api/responder/incidents/:id	Responder, database	Detail of an assigned incident
GET /api/responder/notifications	Responder, database	Assignment notifications
GET /api/admin/dashboard	Administrator, database	System totals and status counts
GET /api/admin/users, GET /api/admin/users/:id	Administrator, database	User list and detail
PATCH /api/admin/users/:id, DELETE /api/admin/users/:id	Administrator, database	Edit or remove a user
GET /api/admin/users/:userId/emergency-contacts	Administrator, database	Contacts of one user
GET /api/admin/emergency-contacts	Administrator, database	Contacts across users with owner search
PATCH /api/admin/emergency-contacts/:contactId	Administrator, database	Correct any contact
DELETE /api/admin/emergency-contacts/:contactId	Administrator, database	Remove any contact
GET /api/admin/notifications	Administrator, database	Notification records
GET /api/admin/incidents/summary	Administrator, database	Counts grouped by status
GET /api/admin/incidents, GET /api/admin/incidents/:id	Administrator, database	Incident list and detail
PATCH /api/admin/incidents/:id/status	Administrator, database	Advance the workflow status
POST /api/admin/facilities, GET /api/admin/facilities	Administrator, database	Create and list facilities
GET /api/admin/facilities/:id, PATCH /api/admin/facilities/:id	Administrator, database	Facility detail and edit
POST /api/admin/rescue-teams, GET /api/admin/rescue-teams	Administrator, database	Create and list teams
GET /api/admin/rescue-teams/:id, PATCH /api/admin/rescue-teams/:id	Administrator, database	Team detail and edit
GET /api/admin/rescue-teams/:id/members	Administrator, database	Team membership list
POST /api/admin/rescue-teams/:id/members	Administrator, database	Add a team member
DELETE /api/admin/rescue-teams/:id/members/:userId	Administrator, database	Remove a team member
POST /api/admin/incidents/:id/assignments	Administrator, database	Assign a team to an incident
GET /api/admin/incidents/:id/assignments	Administrator, database	Assignments of an incident
PATCH /api/admin/assignments/:id	Administrator, database	Advance an assignment status
GET /api/admin/unsafe-reports, GET /api/admin/unsafe-reports/:id	Administrator, database	Review queue and detail
PATCH /api/admin/unsafe-reports/:id	Administrator, database	Verify or edit a report
DELETE /api/admin/unsafe-reports/:id	Administrator, database	Remove a report
GET /api/admin/incidents/:id/risk	Administrator, database	Stored assessments of an incident
GET /api/admin/incidents/:id/nearby-	Administrator, database	Nearby help with coordinates shown

resources		
POST /api/admin/reports, GET /api/admin/reports	Administrator, database	Generate and list reports
GET /api/admin/reports/:id	Administrator, database	Read a generated report
GET /api/admin/reports/:id/export	Administrator, database	Export in PDF, CSV or JSON
DELETE /api/admin/reports/:id	Administrator, database	Remove a generated report
GET /api/health	Public	Service status and database state

Two properties of this table deserve emphasis for the viva. First, ownership is always derived from the JWT identity on the server; incident and contact identifiers in the URL select records, they never confer access, and a user who guesses another account’s identifier receives a not-found response rather than the record. Second, failures are compartmentalised: an unreachable AI service, map provider or weather upstream produces a retryable error for that panel alone, while incident creation, status management and stored-resource lookup continue to work.

4.1.5 Risk Scoring Design

Risk scoring is the only component called “AI” in this project, and the name needs care: the service performs no machine learning, interprets no text and calls no external model. The Node.js backend counts bounded facts from stored records around the incident location — the priority, the incident type, nearby verified and unverified unsafe-area reports, and other nearby active incidents — and the FastAPI service applies fixed arithmetic documented as model raksha-risk-v1. Because free-text descriptions never cross into the scorer, prompt-injection style input cannot influence the result; only the pre-computed numbers matter.

Factor	Contribution	Cap
Priority base	LOW 15, MEDIUM 35, HIGH 60, CRITICAL 80	Always exactly one base value
Incident-type modifier	Safety +0, Disaster +5	Fixed per type
Verified unsafe reports nearby	+6 each	+30
Unverified reports nearby	+2 each	+10
Other active incidents nearby	+3 each	+15
Total score	Sum of the above, clamped to 0–100	Integer only

Level bands follow the score directly: below 25 is LOW, below 50 is MEDIUM, below 75 is HIGH, and anything at or above 75 is CRITICAL. The response carries the score, the level, the model version, the five weighted input factors and the assessment timestamp, and the backend re-validates every field — integer score within range, level within the four enum values, non-empty version, factor array and parseable time — before anything is stored in RiskAssessments. An invalid, unreadable or late response becomes a retryable panel error, and the frontend displays the stored record with the decision-support disclaimer on every view.

4.2 System Interface Design

The RakshaSafe interface gives users and administrators a consistent workspace: a navigation bar with the золотые modules for the signed-in role, a content area of cards, tables with search and pagination, explicit loading skeletons, empty states and error states with retry, and small badges for status, priority and levels. Forms use labelled inputs with inline field errors, destructive actions always ask for confirmation, and phone numbers become tel: links while coordinates become map

links. The layout adapts from mobile to desktop and honours the light and dark themes. The interfaces below are the ones actually implemented in the frontend pages.

4.2.1 Login Interface

The Login Interface is the entry point for every account. It accepts the registered email address or phone number together with the password, authenticates against the backend, and routes the session to the dashboard that matches the account role. The page stays deliberately plain so that signing in remains fast even on weak connections.

Main elements:

- Email-or-phone field for the registered identifier.
- Password field with a show/hide visibility option.
- Sign In button for authentication.
- Create Account link for new users.
- Session-expired notice when redirected after a timeout.

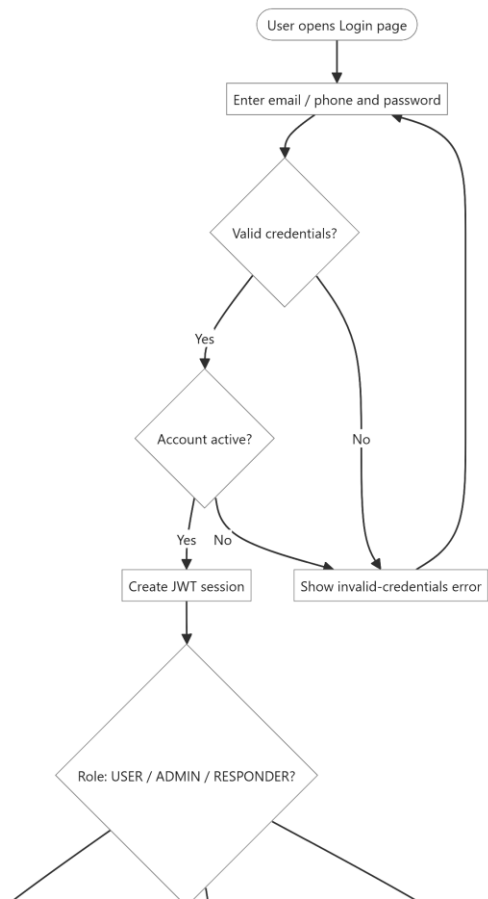
Validation:

- The identifier must be a recognised email address or phone number; blank or malformed input is rejected locally.
- The password must be entered; empty submissions are blocked.
- Invalid credentials are rejected by the backend without revealing which half was wrong.
- Deactivated accounts are refused with a contact-support message.
- Only an authenticated session can leave this page for the application.

Steps:

1. Enter the registered email address or phone number.
2. Enter the password.
3. Click Sign In.
4. The system validates the credentials and verifies the account is active.
5. On success the user is redirected by role: USER to the user dashboard, ADMIN to the admin dashboard, RESPONDER to the responder workspace.

Flowchart 4.2.1: Login



Flowchart 4.2.1: Login

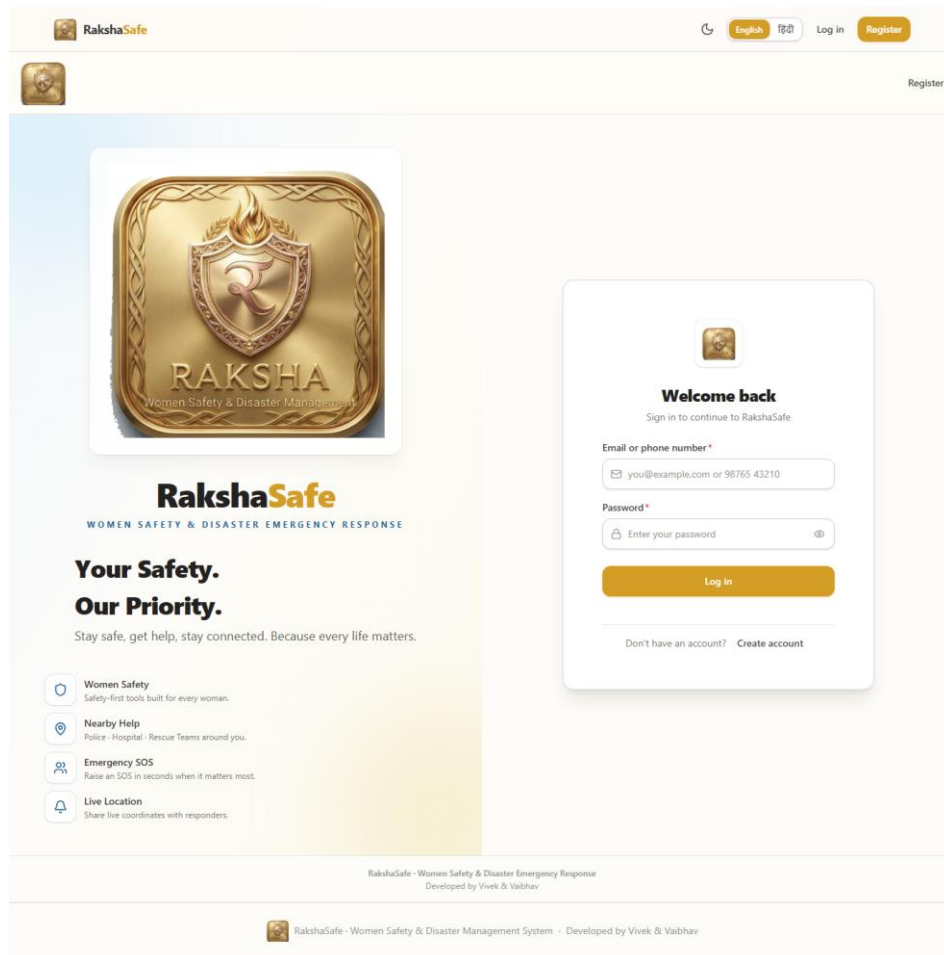


Figure 4.2.1: Login Interface

4.2.2 User Registration Interface

The Registration Interface creates a standard USER account. It collects the minimum profile needed for emergency use — a name, a reachable email and phone number, and a password — and leaves administrative roles to be granted outside this form. Successful registration signs the session in directly, without any email-code step.

Main elements:

- Full-name field.
- Email address field.
- Phone number field.
- Password field with strength guidance.
- Preferred-language option.
- Create Account button and a Back to Sign In link.

Validation:

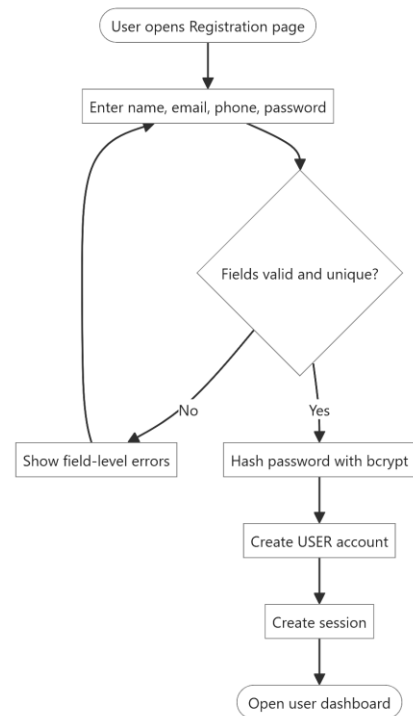
- Name, email, phone and password are all required.
- Email must be valid in format and unique; phone must be a valid number and unique.
- The password must satisfy the minimum-length and complexity rule.
- Duplicate email or phone registrations are rejected with a field-level message.

Steps:

1. Enter the full name.
2. Enter the email address.

3. Enter the phone number.
4. Create a password.
5. Optionally choose the preferred language.
6. Click Create Account.
7. The system validates, stores the hashed password, creates the session and opens the user dashboard.

Flowchart 4.2.2: Registration



Flowchart 4.2.2: Registration

The image shows the 'Create your account' interface of the RakshaSafe application. The header includes the RakshaSafe logo, a language selector (English/Hindi), and links for 'Log in' and 'Register'. The main heading is 'Create your account' with a subtext: 'Create an account to report emergencies and manage emergency contacts safely.' Below this is a warning box: 'Never share your password with anyone.' The form contains the following fields: 'Full name' (with 'Priya Sharma' entered), 'Email' (with 'priya640968@raksha.test' entered), 'Phone number' (with '9964096877' entered), 'Password' (masked with dots), and 'Confirm password' (also masked). A note below the password fields states: 'At least 8 characters. Never share your password.' There is a checked checkbox for 'I agree to the Terms & Conditions and Privacy Policy.' At the bottom of the form is a 'Create account' button and a link for 'Already have an account? Log in'. The footer text reads: 'RakshaSafe - Women Safety & Disaster Management System - Developed by Vivek & Vaibhav'.

Figure 4.2.2: Registration Interface

4.2.3 User Dashboard Interface

The Dashboard is the home screen for a signed-in user. Rather than analytics, it answers three questions at a glance: what are my recent incidents, what state is each one in, and where do I go next. Quick actions lead to SOS reporting, emergency contacts, resources, unsafe-area reporting and notifications, so the dashboard behaves as a hub instead of a dead end.

Main elements:

- Recent-incidents list with category, priority badge and status badge.
- Status summary of the user's own active and resolved incidents.
- Quick actions: Report SOS, Emergency Contacts, Find Resources, Report Unsafe Area.
- Notifications entry with unread indication.
- Shortcuts to profile and language settings.

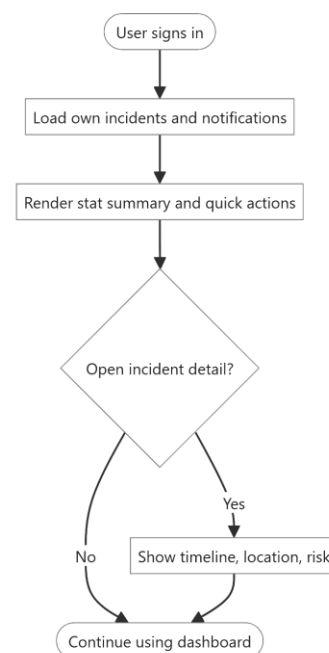
Validation:

- Only the signed-in user's own incidents are listed; no other user's data appears.
- Counts are computed from stored records and never include other accounts.
- Navigation guards keep signed-out visitors on the login page.

Steps:

1. Sign in as a user.
2. Review the recent-incidents list and their statuses.
3. Use a quick action to start SOS reporting, manage contacts, or browse resources.
4. Open any incident for its full detail, history and risk information.

Flowchart 4.2.3: User Dashboard



Flowchart 4.2.3: User Dashboard

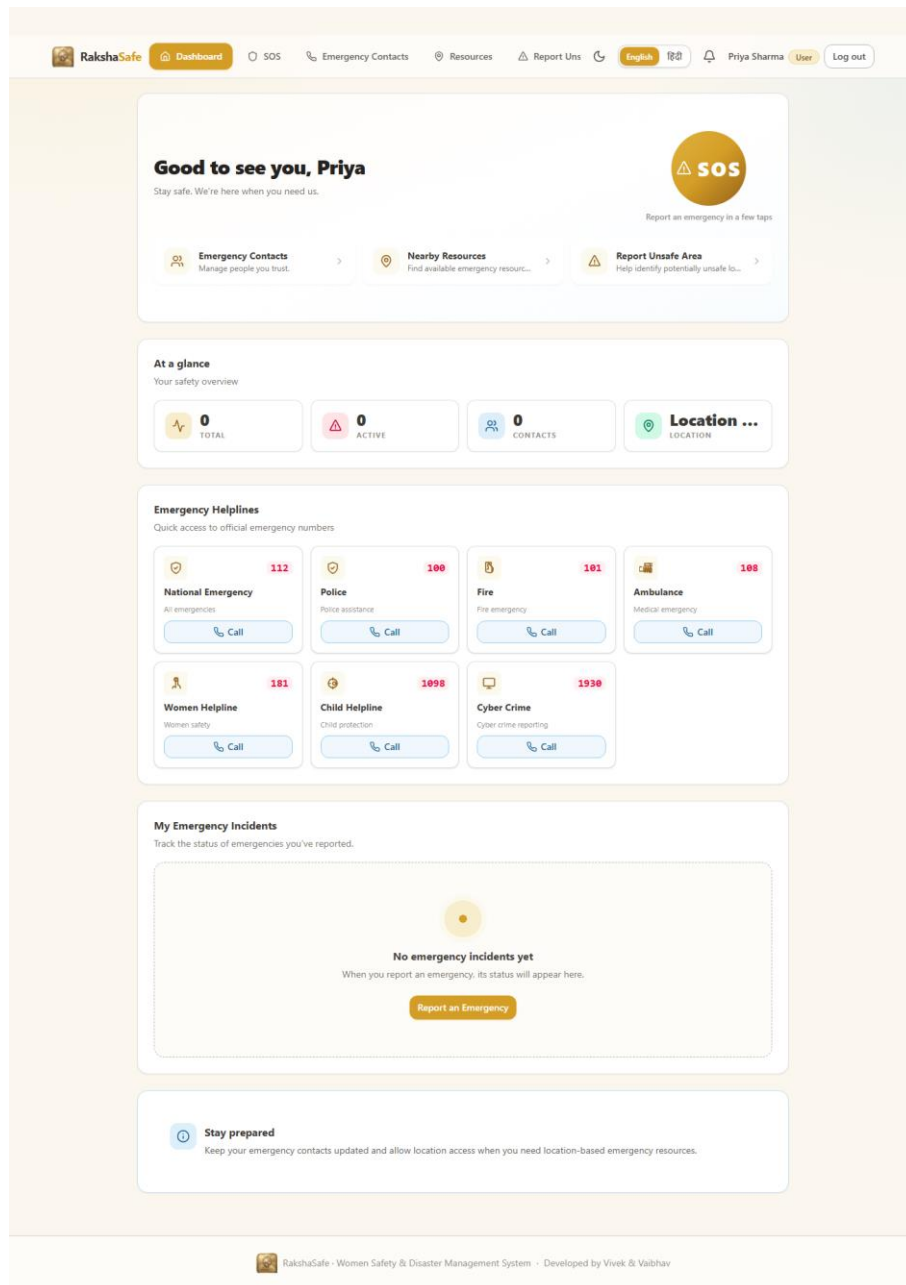


Figure 4.2.3: Dashboard Interface

4.2.4 SOS and Incident Creation Interface

The SOS Interface turns an emergency into a structured incident record in a few guided steps. It asks for the incident type, category, description and priority first, then offers location capture, and finally shows a review screen before submission. Location is helpful but never mandatory: the incident is created with or without coordinates.

Main elements:

- Incident-type selector: Safety or Disaster.
- Category selector: Women Safety, Medical Emergency, Accident, Fire, Flood, Earthquake, Other Emergency.
- Description area for what is happening and who is affected.
- Priority selector: LOW, MEDIUM, HIGH, CRITICAL.
- Use My Location button with permission, timeout and accuracy handling.
- Review screen showing every entered value before submission.

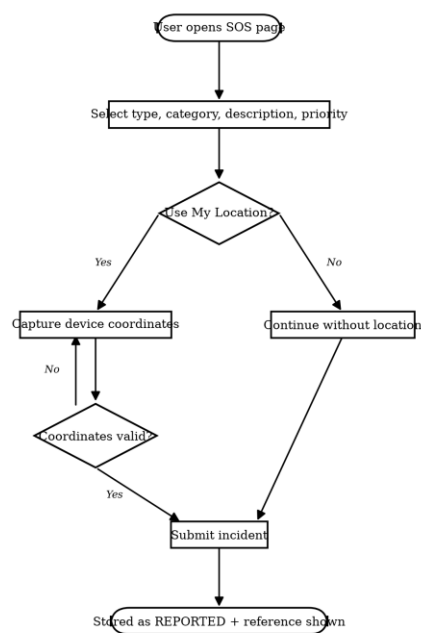
Validation:

- Type, category, description and priority are required; descriptions have a maximum length.
- Coordinates, when supplied, must be valid latitude (-90 to 90) and longitude (-180 to 180).
- Denied or failed geolocation records the incident without coordinates instead of inventing any.
- Double submission is prevented while a request is in flight.

Steps:

1. Open the SOS page and enter type, category, description and priority.
2. Optionally capture the device location or continue without it.
3. Review the entered details on the confirmation screen.
4. Submit the SOS request.
5. The system stores the incident as REPORTED and shows its reference.

Flowchart 4.2.4: SOS Creation



Flowchart 4.2.4: SOS Creation

Figure 4.2.4: SOS Interface

4.2.5 Incident Detail Interface

The Incident Detail Interface is the user's single view of one incident: its facts, its location, nearby help, its risk assessment, any assigned team and its full history. Supplementary sections such as nearby resources, weather and risk load independently, so a failure in one never hides the incident itself.

Main elements:

- Header with category, reference id, status badge and priority badge.
- Description, reported and resolved timestamps.
- Location card with coordinates, accuracy, resolved area and a map link.
- Nearby emergency resources with Call and Directions actions.
- Informational weather card for the incident coordinates.
- AI risk assessment panel with Assess Risk and Try Again actions.
- Assignment card when a team is attached, with the team contact.
- Chronological history timeline of status changes and comments.
- Edit and Delete actions while the incident is REPORTED or ACKNOWLEDGED and unassigned.

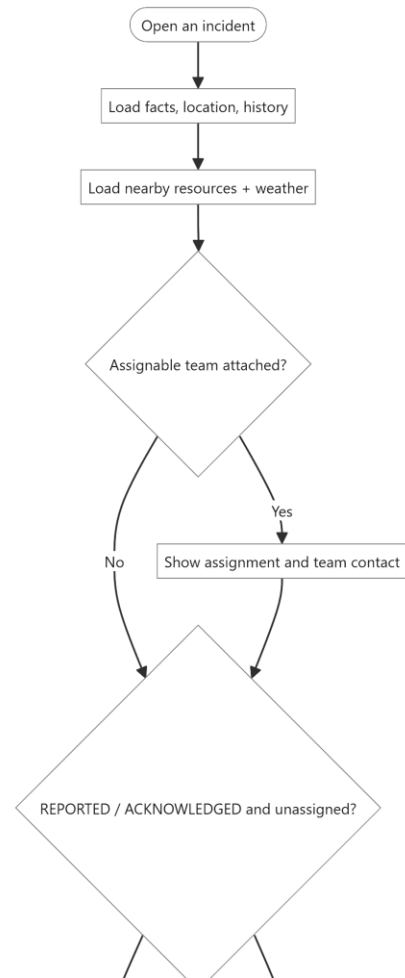
Validation:

- Only the owning user can open this view; other users receive a not-found response.
- Edit and delete are disabled once the incident leaves the early workflow or gains an assignment.
- Map and call links render only from real stored values.

Steps:

1. Open an incident from the dashboard.
2. Read the facts, location and history.
3. Use nearby resources or the team contact directly when needed.
4. Request a risk assessment or retry it when the service was unavailable.
5. Edit or delete the incident while it is still manageable.

Flowchart 4.2.5: Incident Detail



Flowchart 4.2.5: Incident Detail

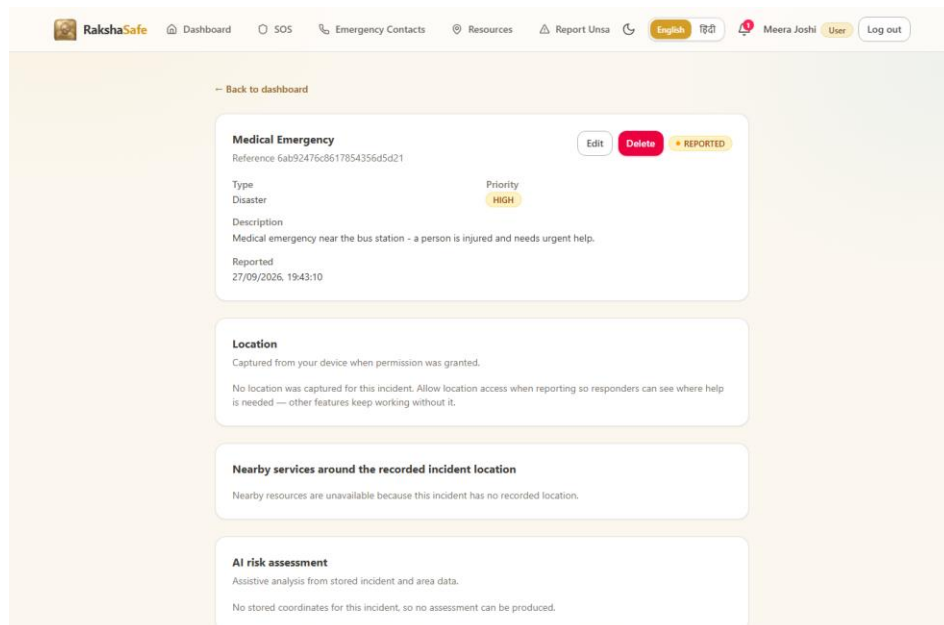


Figure 4.2.5: Incident Detail Interface

4.2.6 Emergency Contacts Interface

The Emergency Contacts Interface lets each user keep the people who should be reachable in an emergency. Contacts belong strictly to their owner: the same stored record later appears to

administrators with its owner attached, but no user ever sees another user's contacts. A primary flag marks the first contact, and per-contact toggles decide SMS and email notification channels.

Main elements:

- Contact list with name, phone, relationship and channel badges.
- Primary-contact marker with a one-primary-per-user rule.
- Add Contact form: name, phone, email, relationship, SMS and email toggles.
- Edit form pre-filled from the stored record.
- Delete confirmation naming the contact before removal.
- Call buttons using tel: links built from the stored phone number.

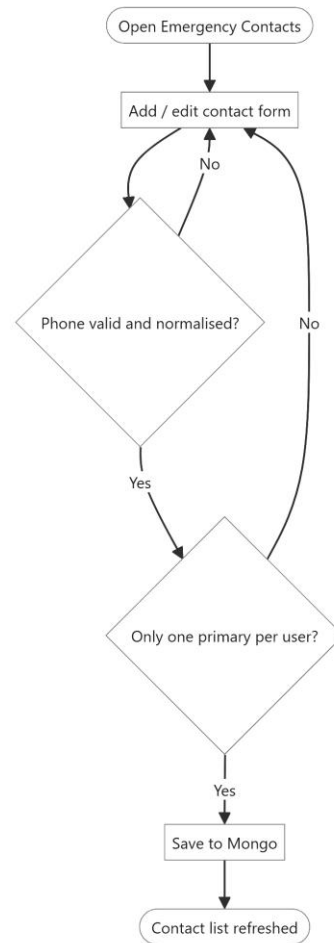
Validation:

- Name and phone are required; email and relationship are optional.
- Phone numbers are validated and normalised to the Indian format before saving.
- Duplicate-primary violations and invalid numbers return field-level errors.
- Ownership is taken from the session; a userId supplied by the client is never trusted.

Steps:

1. Open Emergency Contacts.
2. Click Add and enter name, phone and optional details.
3. Save; the contact appears in the list immediately.
4. Edit any contact to correct details or change channel toggles.
5. Delete with confirmation; the row is removed only after the backend confirms.

Flowchart 4.2.6: Emergency Contacts



Flowchart 4.2.6: Emergency Contacts

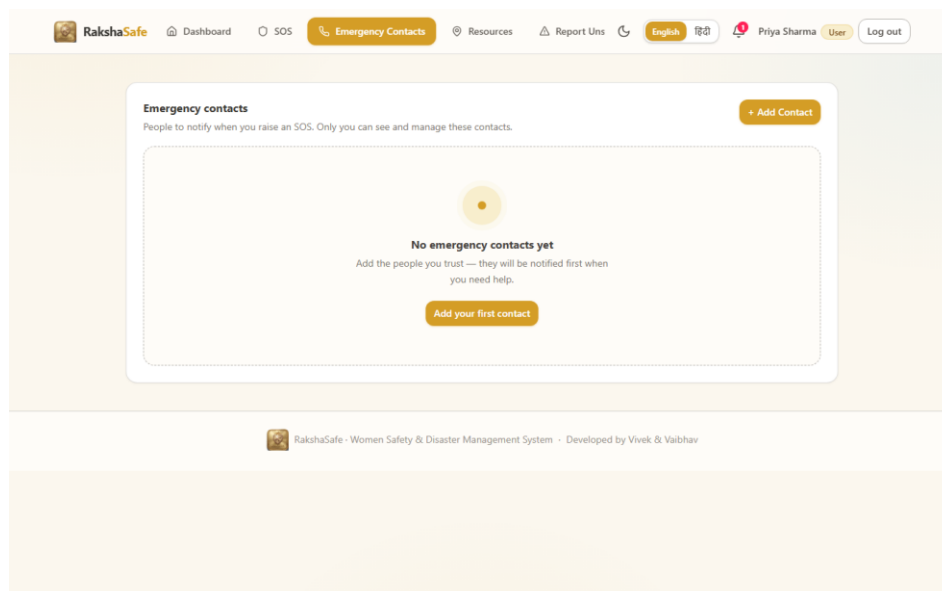


Figure 4.2.6: Emergency Contacts Interface

4.2.7 Emergency Resources Interface

The Resources Interface is a directory of help, not a dispatch console. It searches stored Facilities and active RescueTeams by name, type and operational state, and it can also centre the search on live coordinates to show what is nearest. Every card is honest about what is known and silent about what is not.

Main elements:

- Search box with type and operational-state filters.
- Resource cards with name, facility or team type, phone and address.
- Distance labels computed from real coordinates.
- Call buttons only where a genuine phone number is stored.
- Directions links only where genuine coordinates are stored.
- Source badges distinguishing stored RakshaSafe records from live provider results.

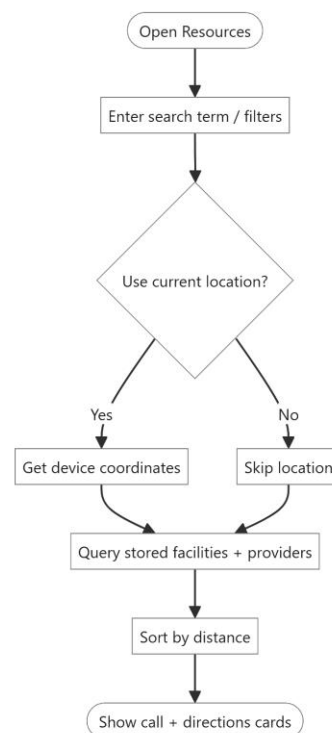
Validation:

- Search text is trimmed and length-limited before querying.
- Radius values are bounded so external providers cannot be overloaded.
- Records without coordinates never receive a fabricated distance.

Steps:

1. Open Resources.
2. Enter a search term or pick a facility type.
3. Optionally centre the search on the current location.
4. Call a resource directly or open walking and driving directions.

Flowchart 4.2.7: Resource Lookup



Flowchart 4.2.7: Resource Lookup

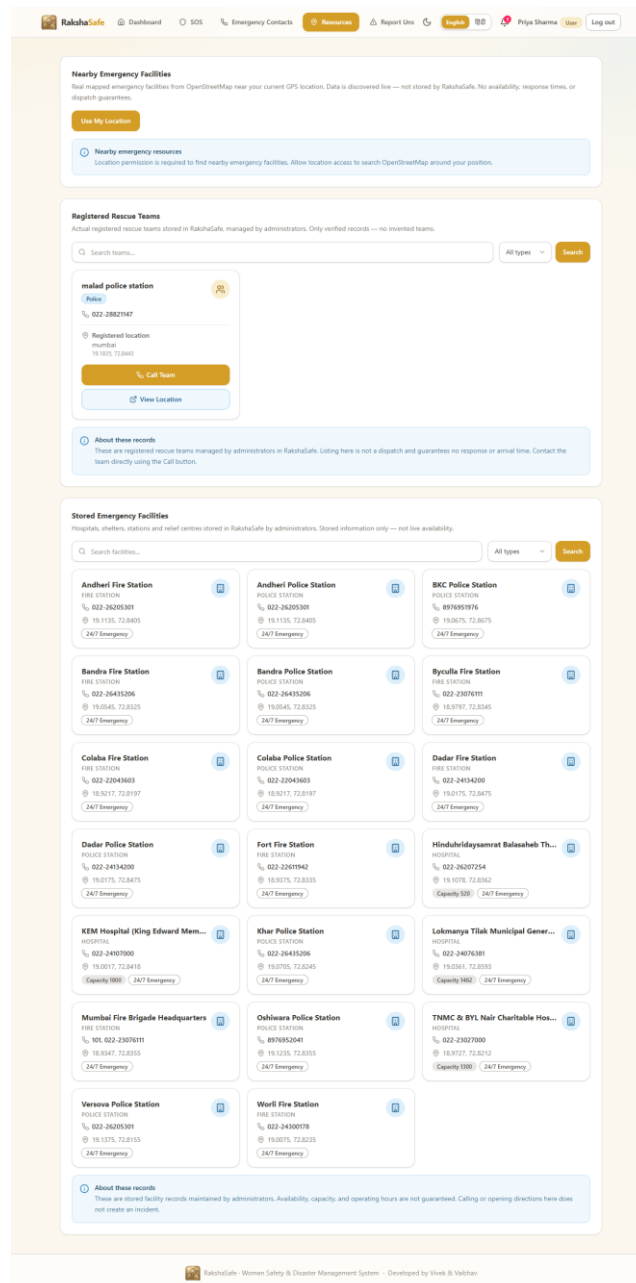


Figure 4.2.7: Emergency Resources Interface

4.2.8 Unsafe Area Report Interface

The Unsafe Area Report Interface lets users flag places that felt unsafe, with a location, a category, a description and a severity. Reports enter the system unverified: they are visible to administrators for review and they contribute to risk scoring only through the verified and unverified counts the backend computes, never as confirmed danger zones on their own.

Main elements:

- Category selector and severity selector.
- Description area for what was observed.
- Location picker using device coordinates or a searched place.
- Own-reports list with verification badges.
- Edit and withdraw actions while the report is still unverified.

Validation:

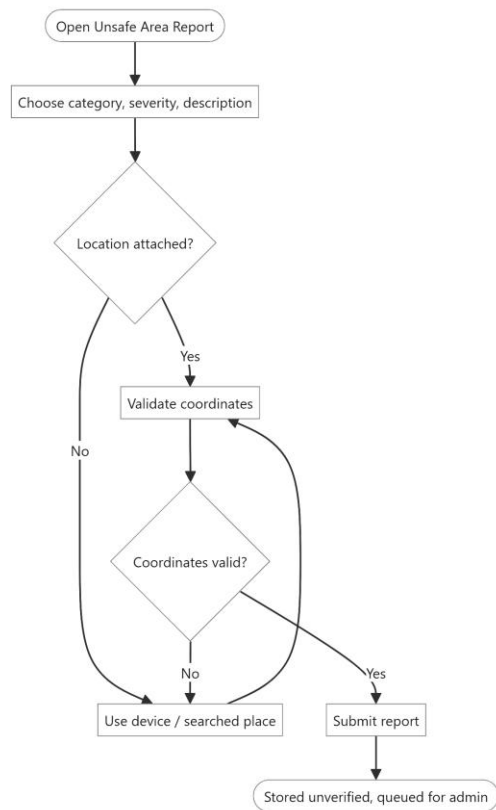
- Category, description, severity and a valid location are required.

- Coordinates are validated before submission; invalid positions are rejected.
- Only the reporting user can edit or withdraw their own report.

Steps:

1. Open the unsafe-area reporting page.
2. Choose the category and severity and describe the concern.
3. Attach the location from the device or search.
4. Submit; the report is stored as unverified and queued for admin review.

Flowchart 4.2.8: Unsafe Reporting



Flowchart 4.2.8: Unsafe Reporting

Figure 4.2.8: Unsafe Area Report Interface

4.2.9 Risk Assessment Interface

The Risk Assessment Interface presents the stored AI assessment for an incident’s location. It shows the backend-computed score out of 100, the LOW, MEDIUM, HIGH or CRITICAL level, the raksha-risk-v1 model version, the weighted input factors and the assessment time, together with the history of earlier assessments. The panel never recalculates anything itself: the backend score is the single source of truth.

Main elements:

- Risk score panel with score, level badge, model version and assessment time.
- Weighted factor list: priority base, type modifier, verified reports, unverified reports, active incidents.
- Assess Risk button for incidents with stored coordinates.
- Try Again button when the AI service was unreachable or timed out.
- Previous-assessments list, newest first.
- Persistent decision-support disclaimer on every result.

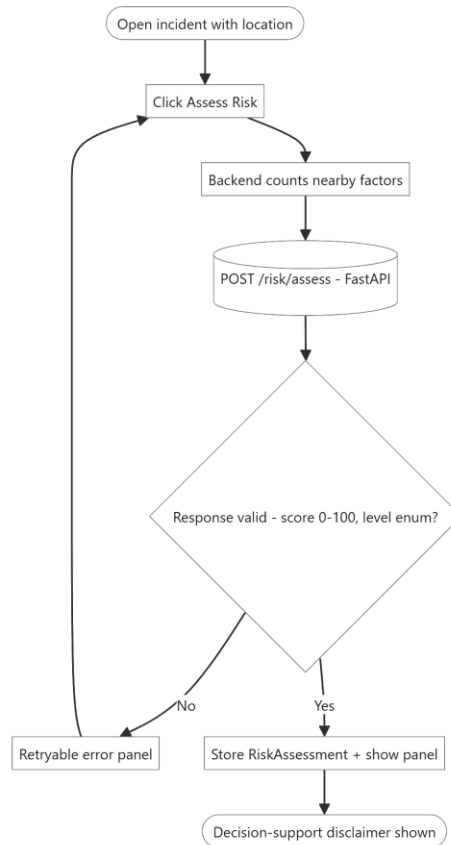
Validation:

- Assessment requires stored coordinates; incidents without a location explain that instead of scoring.
- Backend responses are strictly validated: scores must be integers 0–100 and levels must be one of the four enum values.
- Invalid or unreadable AI responses surface as a retryable error, never as a fabricated score.

Steps:

1. Open an incident that has a location.
2. Click Assess Risk.
3. Read the score, level, factors and time once stored.
4. Revisit the history on refresh; saved assessments persist.
5. Use Try Again if the service was temporarily unavailable.

Flowchart 4.2.9: Risk Assessment



Flowchart 4.2.9: Risk Assessment

RakshaSafe Dashboard SOS Emergency Contacts Resources Ri English हिंदी Raksha Admin Administrator Log out

No team assigned yet
Use the form below to assign a rescue team to this incident.

Assign a team
Rescue team *
Select team
Assign team

Notes (optional)
Staging point, contact person...

AI risk assessment
Assistive analysis from stored incident and area data.

RISK SCORE 80 / 100
RISK LEVEL CRITICAL
Model: raksha-risk-v1 - Assessed: 27/09/2026, 19:40:55

RISK FACTORS

Incident priority: CRITICAL	+80
Incident type: Safety	+0
Verified unsafe-area reports nearby: 0	+0
Unverified reports nearby: 0	+0
Active incidents nearby: 0	+0

Risk level is calculated from the incident priority, incident type and verified safety information available around the reported location.

Decision support only
This score is an assistive risk assessment based on available stored data and does not replace professional emergency judgement.

Figure 4.2.9: Risk Assessment Interface

4.2.10 Notifications Interface

The Notifications Interface keeps the user informed about their own incidents without demanding attention. Entries record what happened, through which channel, and whether delivery succeeded, so the user can distinguish a recorded event from a delivered message.

Main elements:

- Notification list grouped by incident with channel badges.
- Status badges: QUEUED, SENT, DELIVERED, FAILED, NOT_CONFIGURED, UNAVAILABLE.
- Unread markers with a mark-as-read behaviour.
- Provider-response details where available.
- Links back to the related incident.

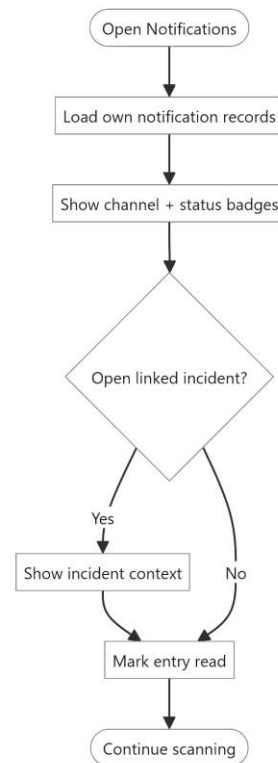
Validation:

- Only the owner's notifications are listed; incident scoping is enforced by the backend.
- Failed or unconfigured deliveries remain visible as records rather than disappearing.

Steps:

1. Open Notifications.
2. Scan the latest entries for incident updates.
3. Open the linked incident for full context.
4. Mark entries read as they are handled.

Flowchart 4.2.10: Notifications



Flowchart 4.2.10: Notifications

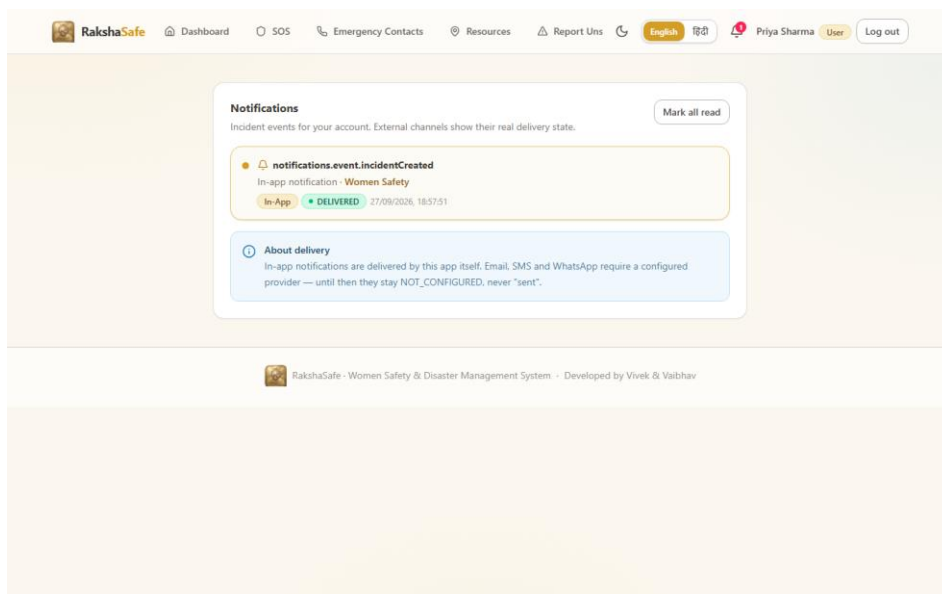


Figure 4.2.10: Notifications Interface

4.2.11 Profile Interface

The Profile Interface shows the signed-in account and lets the user keep reachable details current. Because admins and responders rely on the stored phone number and email, the form treats those fields with the same uniqueness validation as registration.

Main elements:

- Name, email, phone and preferred-language fields.
- Account role and active-state display.
- Save-changes action with inline field errors.
- Sign-out action.

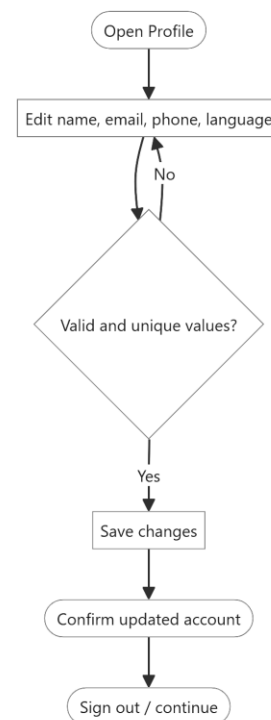
Validation:

- Name cannot be blank; email and phone must stay valid and unique.
- Role and active state are read-only for the user.
- Concurrent edits resolve to the last saved valid state.

Steps:

1. Open Profile.
2. Update name, email, phone or language.
3. Save and confirm the updated details.
4. Sign out when finished on a shared device.

Flowchart 4.2.11: Profile



Flowchart 4.2.11: Profile

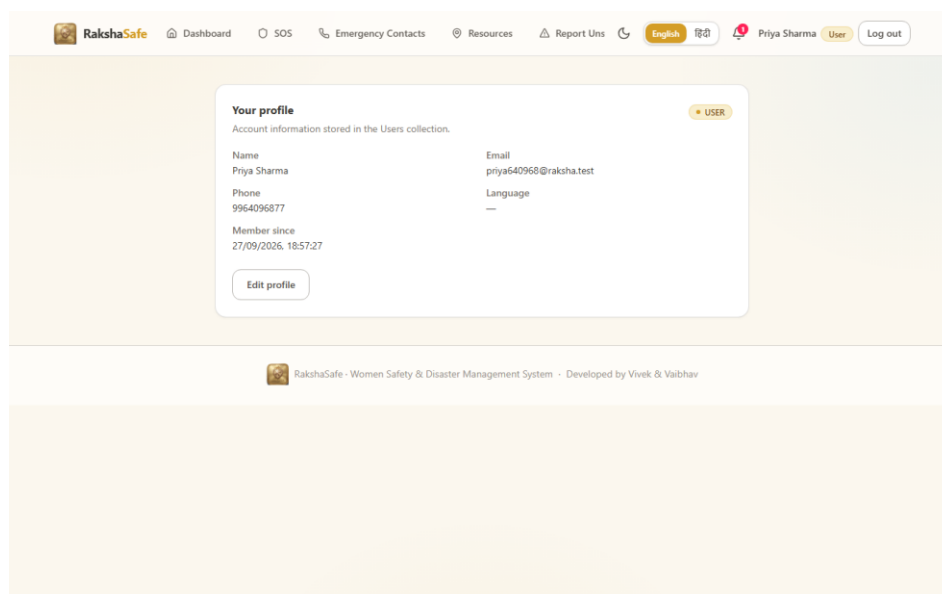


Figure 4.2.11: Profile Interface

4.2.12 Admin Dashboard Interface

The Admin Dashboard summarises the whole system from stored data alone. It reports totals for users and incidents, breaks incidents down by each workflow status, and surfaces facilities, rescue teams, recent incidents and recent administrative activity. Nothing on this screen is predicted; every number is a count of records.

Main elements:

- Total users and total incidents counters.
- Incidents-by-status breakdown: REPORTED, ACKNOWLEDGED, ASSIGNED, IN_PROGRESS, RESOLVED, CLOSED, CANCELLED.
- Active-incident and recent-incident lists.
- Facility and rescue-team availability counts.
- Recent activity feed drawn from AdminLogs.

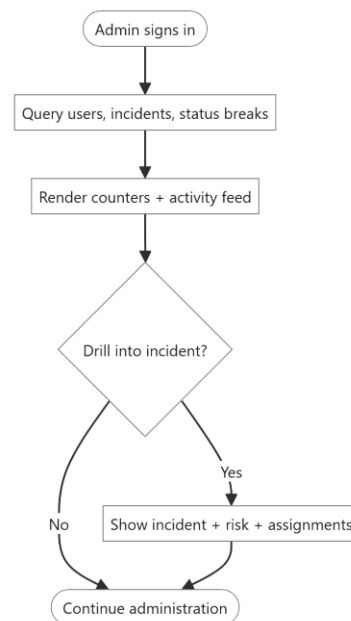
Validation:

- The dashboard requires the ADMIN role; other roles are redirected away.
- All figures derive from database queries at load time.

Steps:

1. Sign in as an administrator.
2. Review totals and the by-status breakdown.
3. Drill into recent incidents or activity for detail.

Flowchart 4.2.12: Admin Dashboard



Flowchart 4.2.12: Admin Dashboard

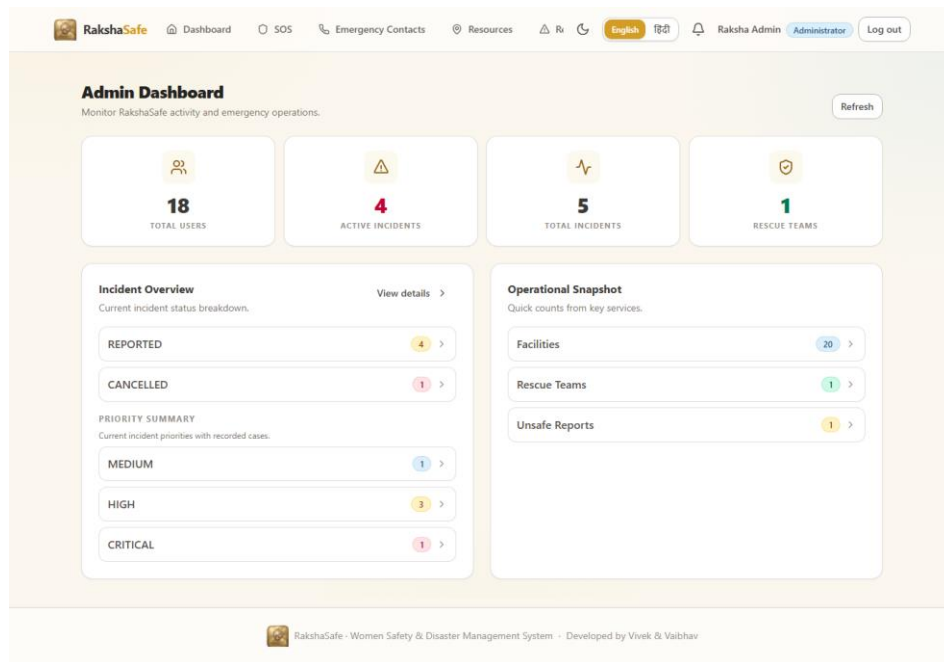


Figure 4.2.12: Admin Dashboard Interface

4.2.13 Admin Incident Management Interface

This is the administrator's working view of emergencies. The list supports status, priority, type and text filters with pagination; the detail view combines the reporter, the captured location, the status timeline, rescue assignments and the stored risk assessments. Status changes and assignments happen here, each recorded with who did it and when.

Main elements:

- Filterable incident table with search across category and description.
- Detail header with category, reference, status and priority.
- Reporter card with name, email and callable phone.
- Location card with coordinates, accuracy, resolved area and map link.
- Status-update form constrained to the legal workflow transitions.
- Team-assignment form with active-team options and notes.
- Assignment-status editor following ASSIGNED → EN_ROUTE → ON_SCENE → COMPLETED or CANCELLED.
- Read-only risk panel showing the same stored assessments the user sees.

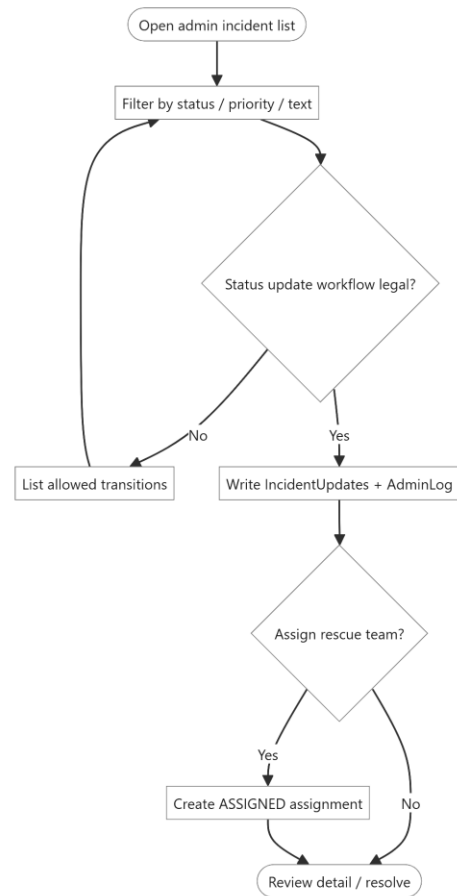
Validation:

- Status transitions outside the documented workflow are rejected with the allowed options listed.
- Only registered, active teams can be assigned.
- Every status change writes an IncidentUpdates record and an AdminLog entry.

Steps:

1. Open the admin incident list and apply filters.
2. Select an incident to review reporter, location, history and risk.
3. Update the status with an optional comment.
4. Assign a rescue team or advance an assignment's status.
5. Resolve, close or cancel the incident as the real situation requires.

Flowchart 4.2.13: Admin Incidents



Flowchart 4.2.13: Admin Incidents

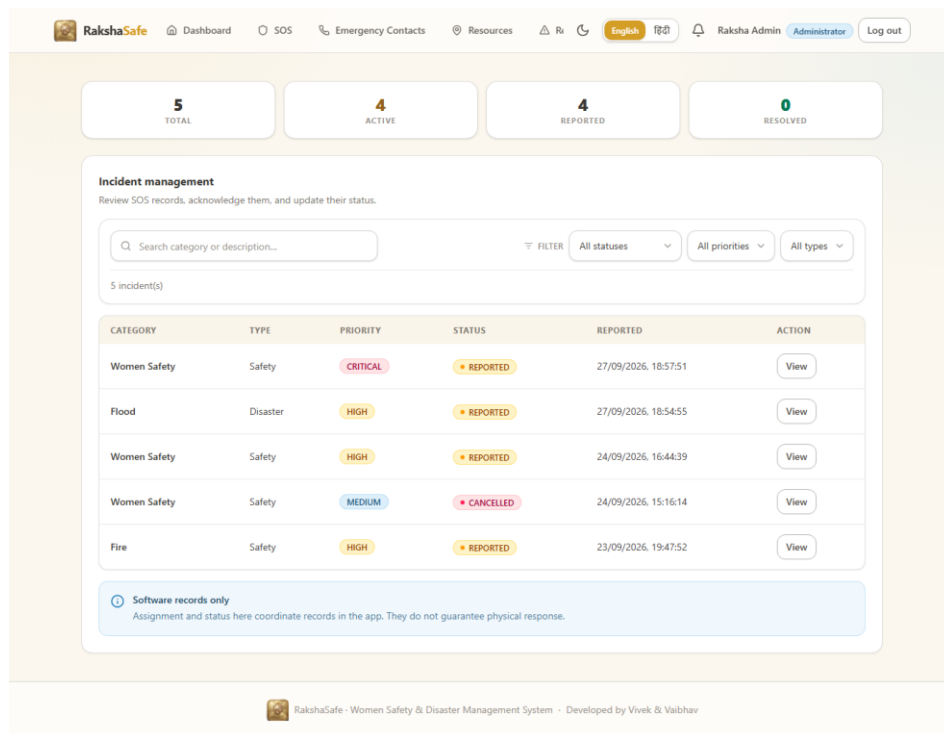


Figure 4.2.13: Admin Incident Interface

4.2.14 Admin Masters Interface

The remaining admin screens manage the master records the rest of the system reads: user accounts, any user's emergency contacts, facilities, rescue teams and their members, unsafe-area report

verification, risk zones, generated reports and the activity log. Each screen follows the same pattern — search, paginated table, view and edit, and confirmation before deletion.

Main elements:

- User management with search, activation control and per-user detail tabs.
- Emergency-contacts browser across all users with owner-name search and Call, Edit and Delete actions.
- Facility management with type, phone, capacity, hours and operational state.
- Rescue-team management with type, contact, specialisations, members and base location.
- Unsafe-report verification queue with verify, edit and delete.
- Risk-zone records with name, level, boundary geometry and active state.
- Report generation in PDF, CSV or JSON with stored snapshots and expiry.
- Admin-log viewer recording administrator, action, target, details, IP and time.

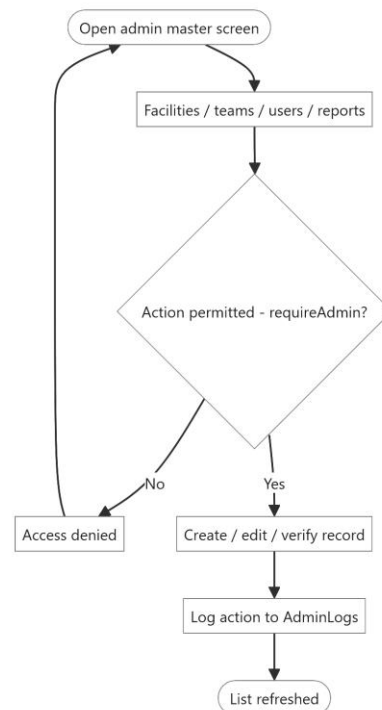
Validation:

- All screens require the ADMIN role in addition to authentication and database availability.
- Phone, email and coordinate rules match the user-side validation.
- Deletions require confirmation and are themselves logged.

Steps:

1. Open the required master screen from the admin navigation.
2. Search or filter to the record.
3. View, create, edit or verify the record as the screen allows.
4. Confirm any deletion and verify the change in the list.

Flowchart 4.2.14: Admin Masters



Flowchart 4.2.14: Admin Masters

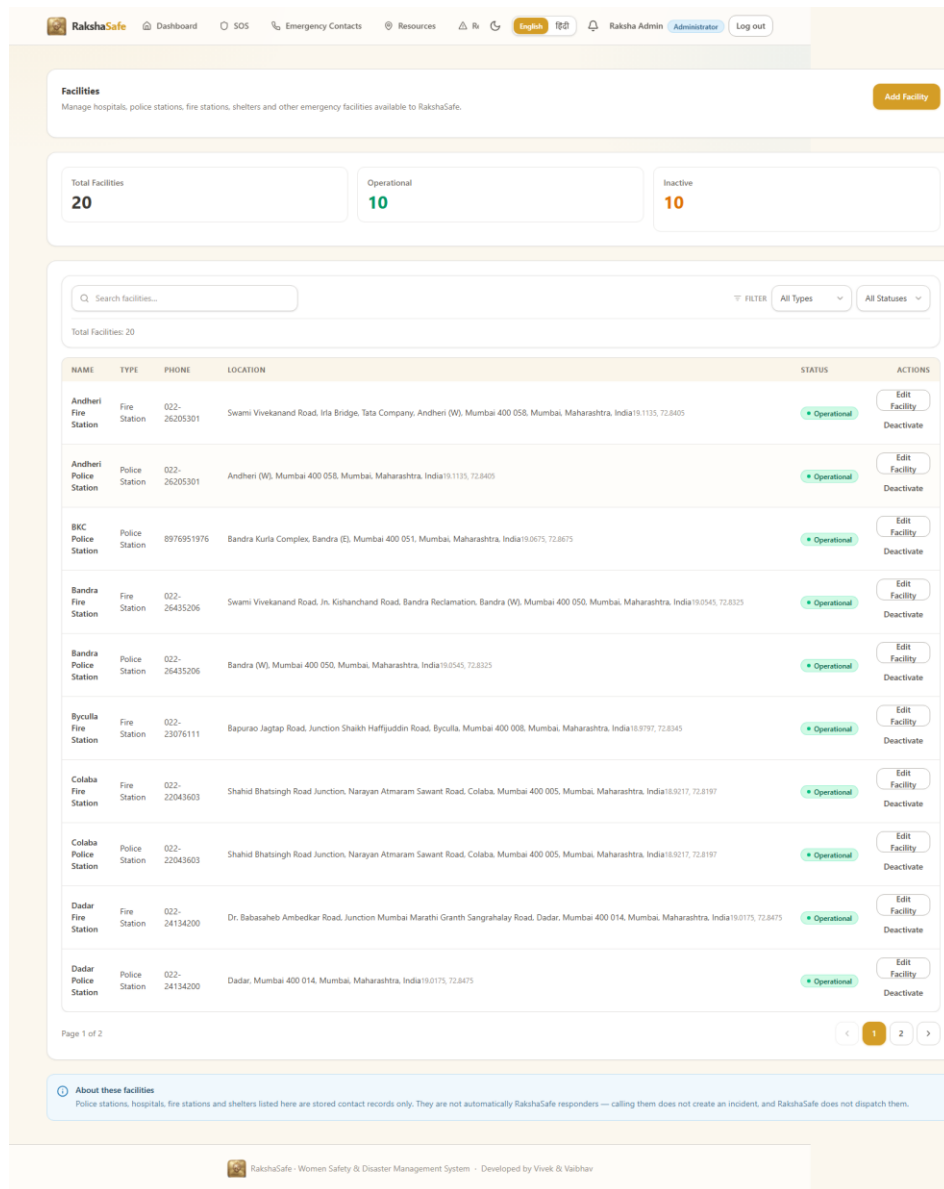


Figure 4.2.14: Administration Interface

4.3 Deployment and Security Design

RakshaSafe is deployed as a small set of cooperating services rather than a single process. This section records how those services are arranged and how the system protects accounts, incidents, locations, contacts and risk information in transit and at rest.

4.3.1 Deployment Design

Development and small-scale hosting run four processes: the Vite frontend, the Express backend, the FastAPI AI service and MongoDB. The frontend talks only to the backend over REST; the backend alone talks to MongoDB and to the AI service. No browser code ever contacts the database directly, and no API key is required for the normal Open-Meteo and OpenStreetMap enrichment endpoints.

Main Services

1. Frontend: runs the React and TypeScript web application built with Vite and serves the user and admin interfaces.
2. Backend: runs the Node.js and Express.js application on port 5000 and exposes the REST API under /api with a health endpoint.

3. AI Service: runs the Python FastAPI application on port 8000 with the deterministic risk endpoint POST /risk/assess.
4. MongoDB: persists the fifteen collections and enforces schema validation; it runs locally at mongodb://127.0.0.1:27017 or at a configured remote URI.

Deployment Steps

1. Prepare the Node.js 20+, Python 3.12+ and MongoDB prerequisites.
 2. Copy each service's .env.example to .env and set the database URI, JWT secret and ports.
 3. Install backend and frontend dependencies and the AI service requirements.
 4. Start MongoDB and confirm connectivity through the backend health endpoint.
 5. Start the AI service and confirm its health endpoint.
 6. Start the backend API and then the frontend development server.
 7. Verify login, SOS submission, location capture, resource lookup and risk assessment end to end.
 8. Serve the production frontend build behind the same backend origin for same-origin API calls.
- Each service keeps its own environment file copied from the corresponding .env.example, so ports, the database URI, the JWT secret and provider settings differ between developers' machines without touching the source. The default development ports are fixed by convention and appear throughout the run instructions:

Service	Default port	Notes
Frontend (Vite)	5173	Development server with /api proxy to the backend
Backend (Express)	5000	REST API under /api; health at /api/health
AI service (FastAPI)	8000	Risk endpoint and interactive docs at /docs
MongoDB	27017	Local default mongodb://127.0.0.1:27017 or a remote URI

Figure 4.3: RakshaSafe Deployment View

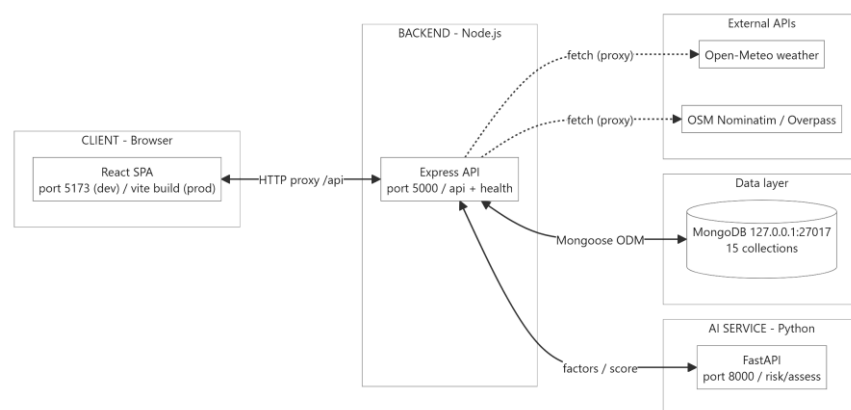


Figure 4.3: RakshaSafe Deployment View

4.3.2 Security Design

Security in RakshaSafe is layered: authentication proves who is calling, role checks prove they may perform the action, ownership checks prove the record is theirs, and validation proves the payload is well formed. Each layer is enforced on the backend, because anything checked only in the browser can be bypassed.

Authentication

- JWT bearer tokens authenticate every protected request; missing or expired tokens are rejected.
- The middleware reloads the account from the database when reachable, so deletions, deactivation and role changes take effect immediately.
- Database-independent enrichment routes stay available under token-only authentication when MongoDB is down.

Password Security

- Passwords are hashed with bcrypt before storage and compared in constant time on login.
- Plain-text passwords are never stored and never returned by any API.

Authorisation and Data Protection

- Routes are guarded by `requireAuth`, `requireAdmin` and `requireResponder` together with `requireDb` where stored data is needed.
- Users reach only their own incidents, contacts, reports, notifications and locations; incident and contact identifiers from the client are treated as untrusted and re-checked against ownership.
- Admin-only data such as other users' contacts, all incidents and admin logs is unreachable to USER and RESPONDER roles.
- Role values come from the database, never from client claims.

Validation and Secret Management

- ObjectIds, enums, phone numbers, coordinates, radius values and pagination parameters are validated with bounded limits before any query or external call.
- Uploaded and searched inputs are length-limited and sanitised; external responses are validated before display.
- JWT secrets, database URIs and provider keys live in server-side environment files; the frontend carries no secrets.

4.4 Chapter Summary

The system design of RakshaSafe records how an emergency travels through the software: a signed-in user reports an SOS incident with an optional real location, the backend validates and stores it, nearby facilities and rescue teams become directly callable, administrators advance the documented status workflow and record rescue assignments, the deterministic `raksha-risk-v1` service scores the location on request, notifications track each lifecycle event, and every administrative action is logged. Fourteen interface groups cover the user journey from login to profile and the admin journey from dashboard to reports, all rendered responsively in both themes. Deployment keeps the web application, the API, the AI service and MongoDB as independent processes, while JWT sessions, bcrypt passwords, role and ownership enforcement and strict validation protect the data. The design claims no automatic dispatch, no guaranteed response and no predicted outcomes: it is a careful record-keeping and decision-support system for real emergencies.